→ NGINX – ~ Techworld With Nana

1) As Web Server -

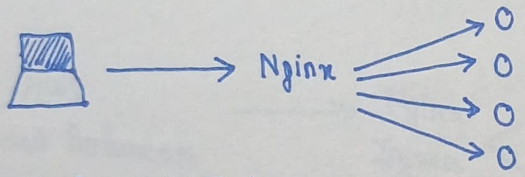
Browser ⇄ HTTP Response ⇄ Web Server (NGINX) - High perf web server

Primary fn. to serve webpages to client browser.

- Web Server is physical machine / s/w running on machine, serves pages to client when requested.

2) As Load Balancer -

Requests ↑, setup multiple servers.

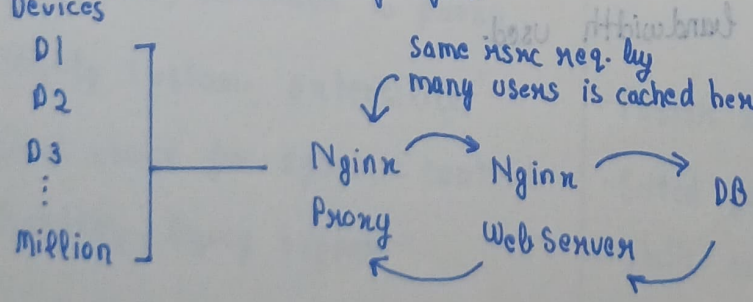


- Dist. incoming traffic to multiple backend servers.
- To balance load, improve performance & provide redundancy.

3) As Proxy Server - Intermediary server that forwards clients requests to other servers.

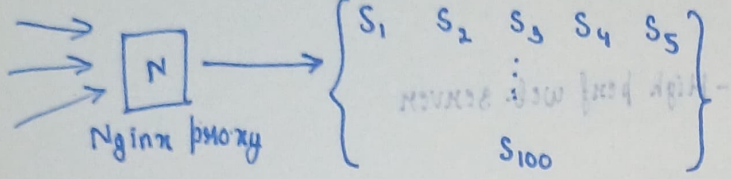
- Load Balancer algorithms - Least Requests, Round robin, Weighted round robin, Weighted least connect^n, ip hash.

4) Caching - Core feature of Nginx



- Fetching from DB for every request from server results in slow response time.
- Copy of static responses stored for quick response.

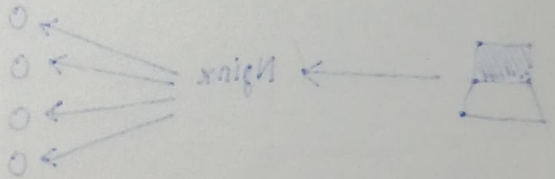
5) Reducing attack surface -



If we expose the 100s of servers to internet, high attack surface, a hacker can easily break into 1 of the server & Boom. → Better

Put one server, one entry point, reduces attack surface.

- Consolidated Security
- Min. exposure
- Centralized logging & security
- Central access control.

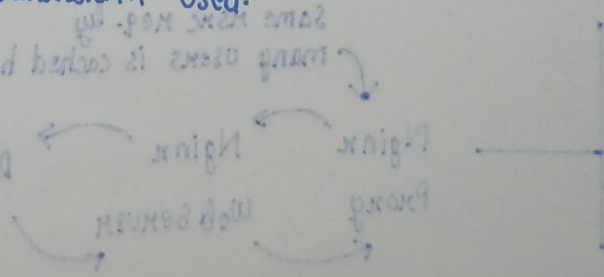


- Only one server publicly available.

→ SSL / Terminat^{n} Offloading -

- Handles SSL / ~~en~~ TLS encrypt^{n} & decryption.
- Web Server decrypts message itself.

→ Compression - Compress media files before sending response, reducing amt. of bandwidth used.

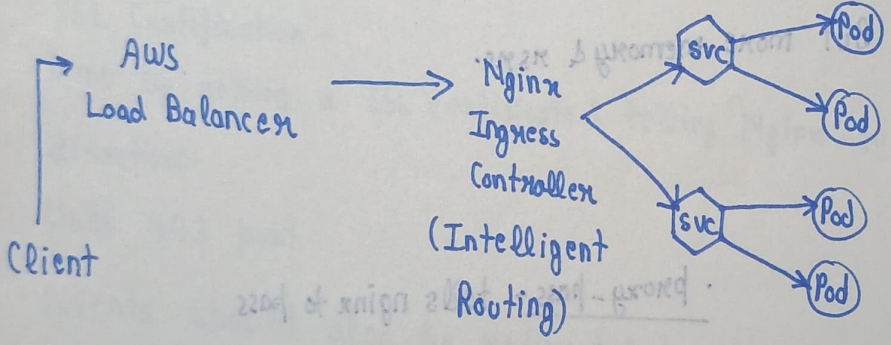


· Nginx Config -

Nginx performs several tasks, which nginx proxy does which task, is configured by a straightfwd nginx.conf file.

*Refer docs

6) K8s ingress controller - Most popular, this similarly like a * reverse proxy, sends a request to right service inside a K8s cluster.



· Pulumi - Abstract^{n} of ingress control setup, can use lang we use & love.

→ Apache vs Nginx -

Apache	Nginx
· Basic WS, extended to proxy	· Faster & more lightwt
· Highly Custom., extensible	· Suited for high-perf. env. & serving stutic content
· Good choice for dynamic content handling, legacy support.	· Simple Config.

- worker_processes - controls how many ||el processes Nginx spawns to handle client requests.
- Worker Process - Instead of using new process for every incoming connection, Nginx uses worker processes, that handle many connect^n using single-thread event loop.
- Each one handles request independently.
- Tuned acc. to server H/w & expected load.

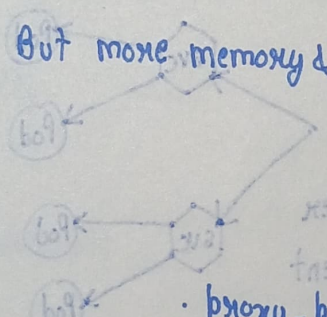
≈ auto

- worker_connections = Per worker processes, how many simultaneous connections can be opened. High value, more connect^n.
- default = 512

But more memory & rsrc.

1 worker → 512 conn

2 worker → 1024 conn



→ Basic Config -

```
http {
    server {
        listen 8080;
        server_name localhost;
        location / {
            root /var/www/html;
            index index.html;
        }
    }
}
```

- proxy_pass - tells nginx to pass req. to another server, acting as reverse proxy.

- Upstream - server that nginx forwards request to. Based on flow of data.
- Upstream server - Traffic going from client to higher level infra.

Downstream server - Traffic going back to client.

- When nginx acts as reverse proxy, servers never know the src of incoming req., only nginx sees it,
  and we need to send this info to servers, for monitoring, logging etc.
  client_ip-address
- Internal backend server not exposed to internet. Only nginx port is kept open.
- Need to tell browser the file type of response, for usage & rendering.
  mime types.

⇒ Directives - Actual instn or cmd that tell Nginx what to do.

⇒ Contexts - Group of related directives that apply to certain part of config.

- SSL Certification -
  Done by getting a SSL certificate & telling Nginx to use that certificate by directives.
  Uses 443 port.
- Clients should also be redirected to https domain even if they don't explicitly type it.